COSC204 Computer Systems

Lecture 19: CPU Scheduling

Zhiyi Huang

School of Computing, University of Otago

Overview

Last lecture:

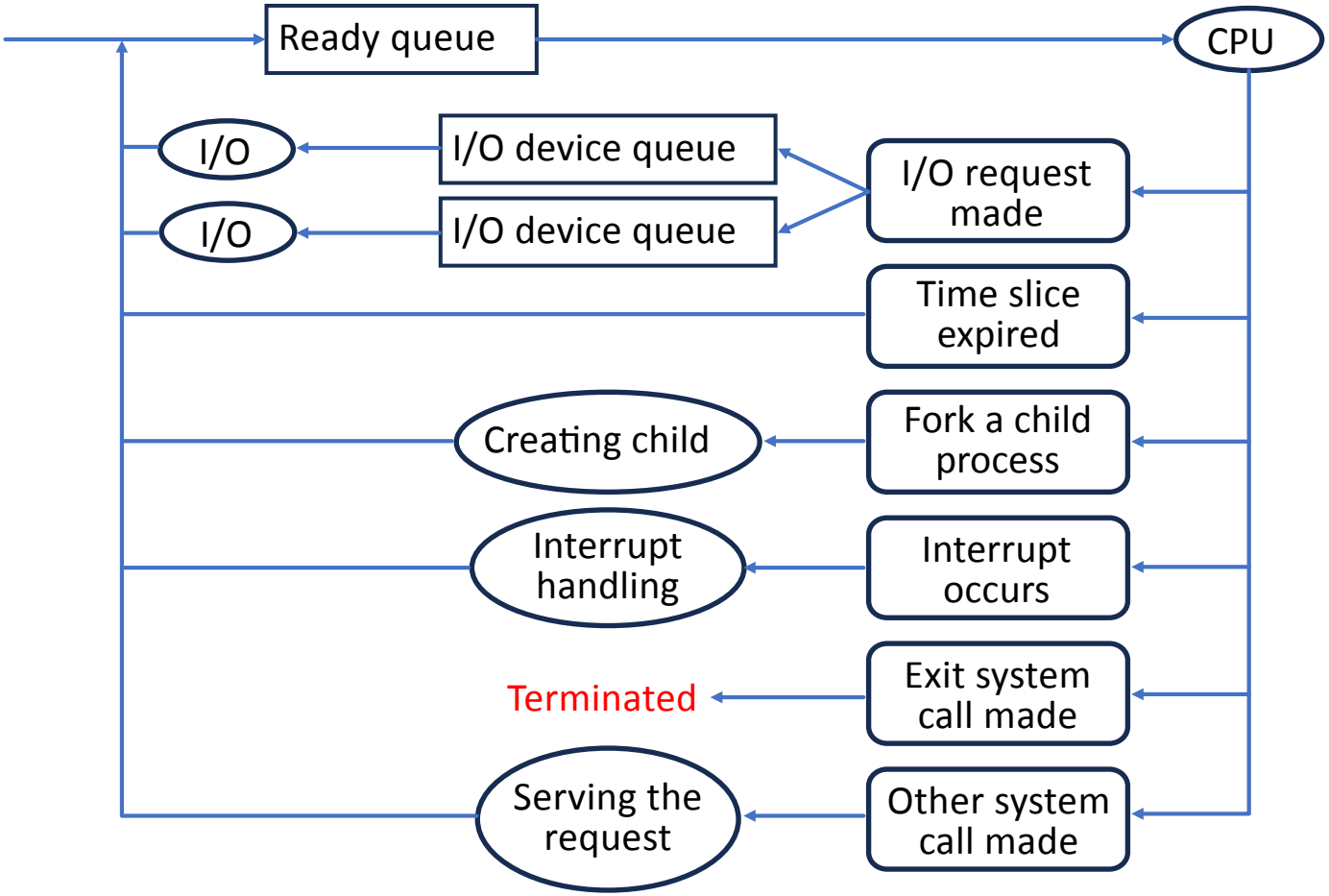
- Shared memory for processes
- Threads

This lecture:

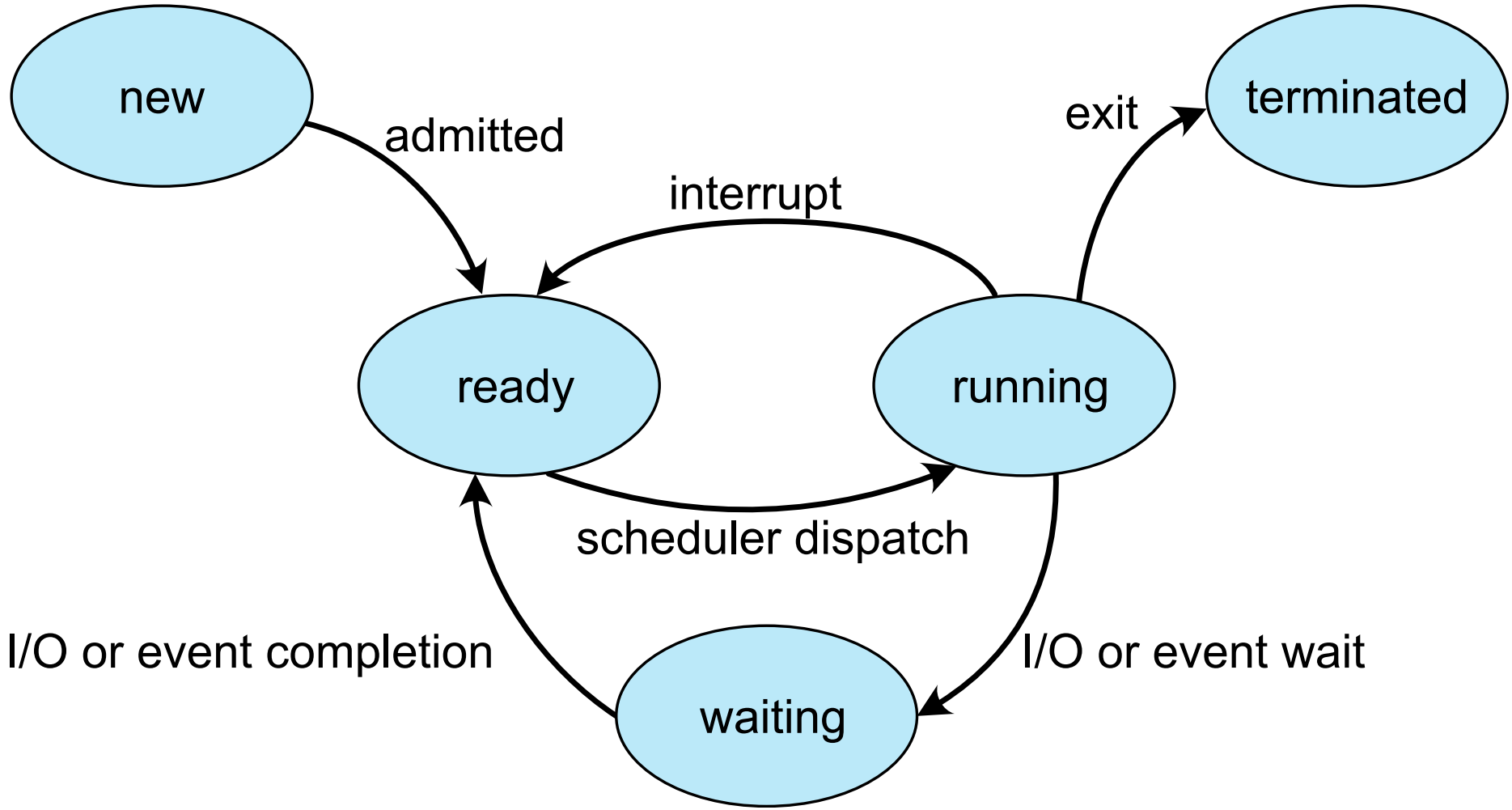
- Criteria for scheduling algorithms
- Some scheduling algorithms: first-come-first-served, shortest-job-first, priority scheduling, round-robin scheduling

Recap: the OS performs CPU scheduling

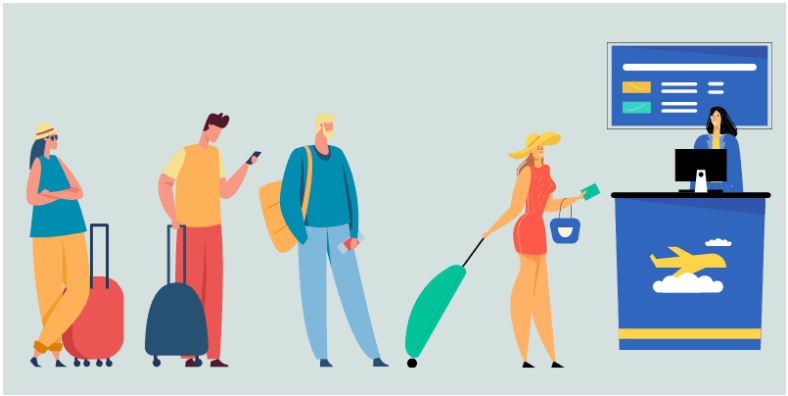
A CPU scheduler is the kernel code that determines how to move processes between the ready queue and the CPU.



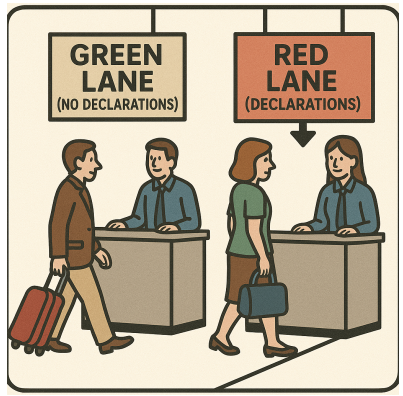
Any OS process will be in one of many states



Four example scheduling algorithms we will explore



First Come First Served



Shortest Job First



Priority Scheduling

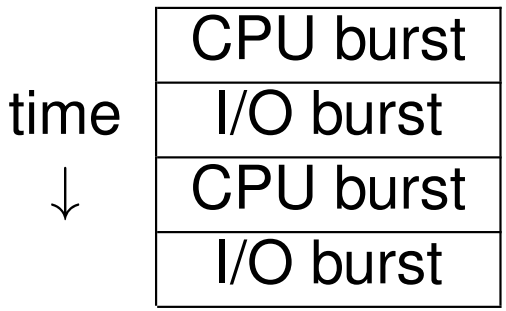


Round Robin

CPU schedulers' goal 1: keep the CPU busy

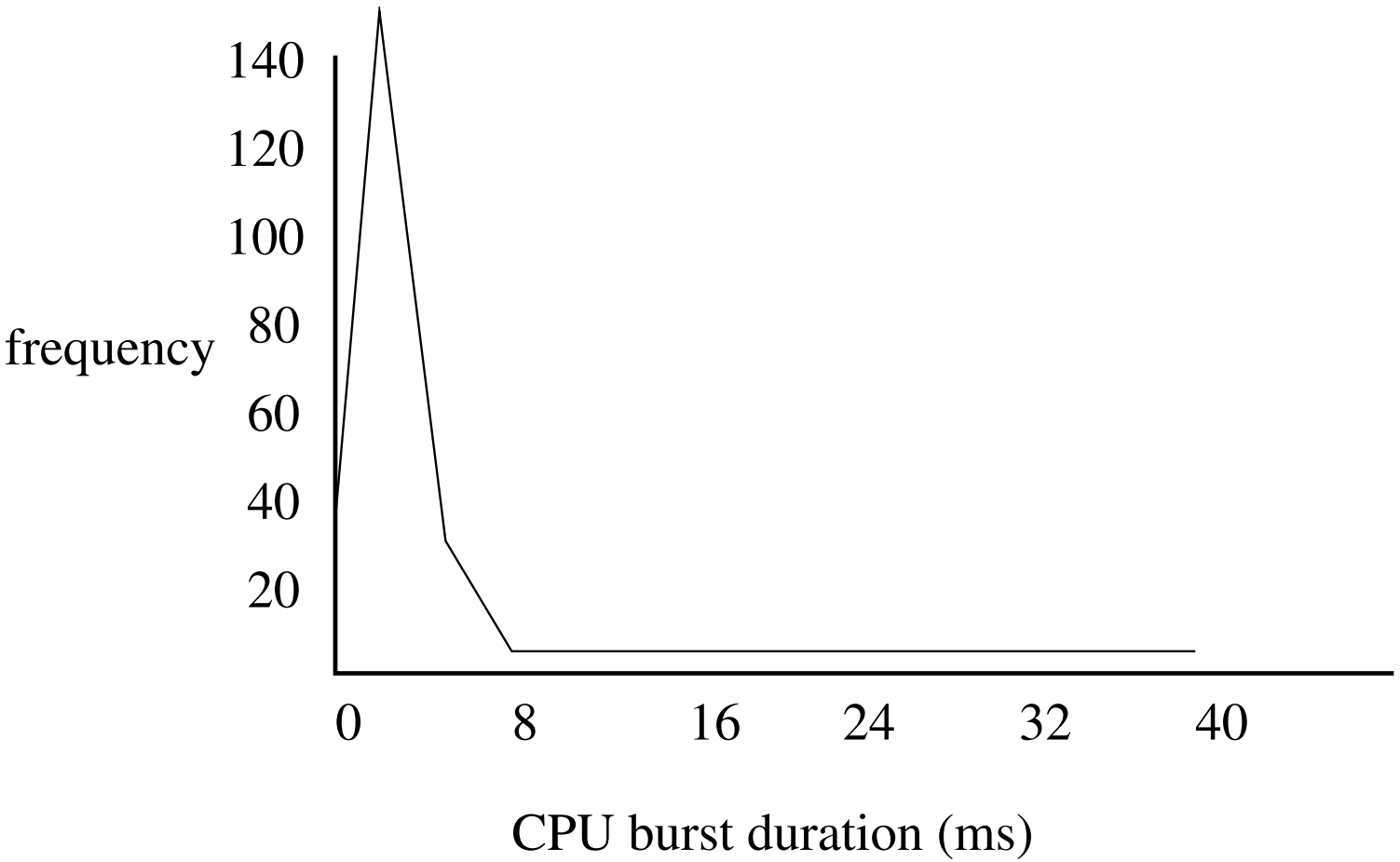
A key goal of CPU scheduling is to *keep the CPU busy* (i.e., achieve high CPU utilisation). Processes are shifted off the CPU when they're waiting for an event.

- If processes never had to wait for events, then scheduling would not increase CPU utilisation.
- However, most real-world processes exhibit a cycle of CPU burst and I/O burst phases.



How long is a CPU burst?

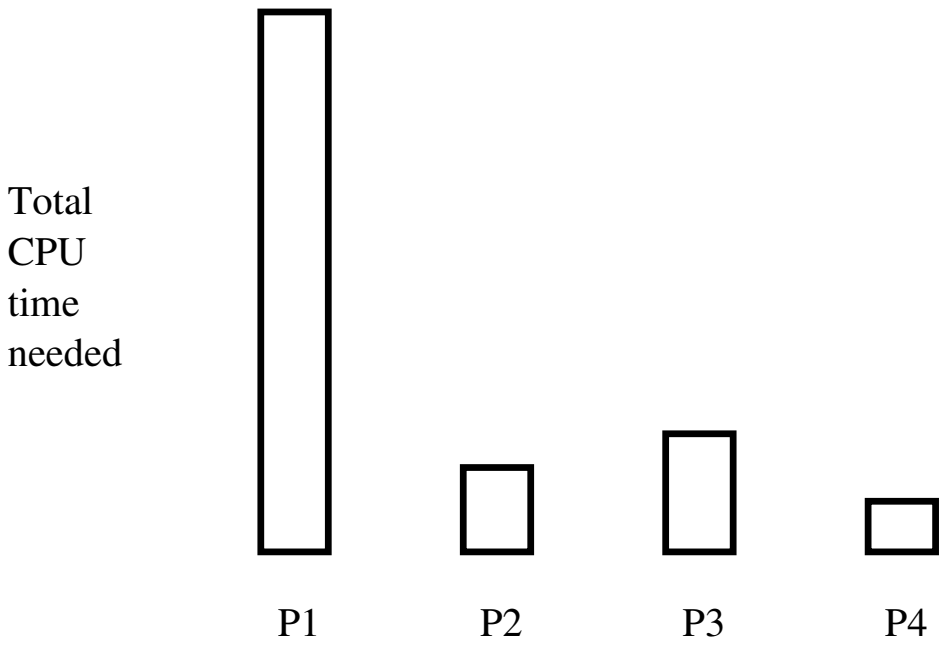
This is the kind of frequency curve we can expect:



CPU schedulers' goal 2: fair sharing of the CPU

Effective scheduling algorithms will ensure that processes do not have to spend too much time *waiting* for the CPU (fairness).

- Even if the CPU is always busy, executing processes in different orders can change the average amount of time a process spends queueing for the CPU.

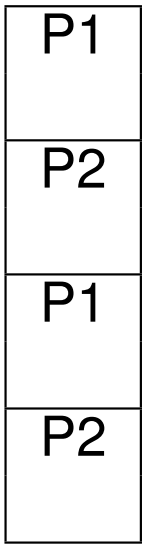


CPU schedulers' goal 3: responsive processes

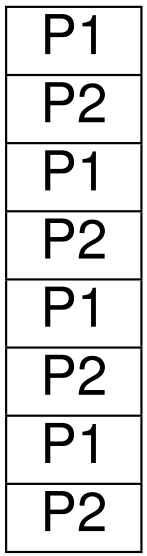
CPU schedulers should aim to make interactive processes *respond* quickly (responsiveness).

- Minimising how long a process spends waiting for the CPU *in total* does not effect responsiveness for an interactive process;
- Instead we want to minimise how long on average it waits *in between* visits to the CPU.

Slow CPU switching
(Less responsive)



time
↓



Fast CPU switching
(More responsive)

Criteria for CPU scheduling algorithms

- CPU utilisation: the percentage of time that the CPU is busy.
- Throughput: the number of processes that complete per time unit.
- Turnaround time (for a single process): the length of time from when the process was submitted (arrived) to when it is completed.
- Waiting time (for a single process): the total amount of time the process spends waiting for the CPU.
- Response time (for a single process): average time from the submission of a request to a process until the first response is produced.

Terminology: (non-)preemptive scheduling

Four situations in which CPU scheduling decisions fit conveniently:

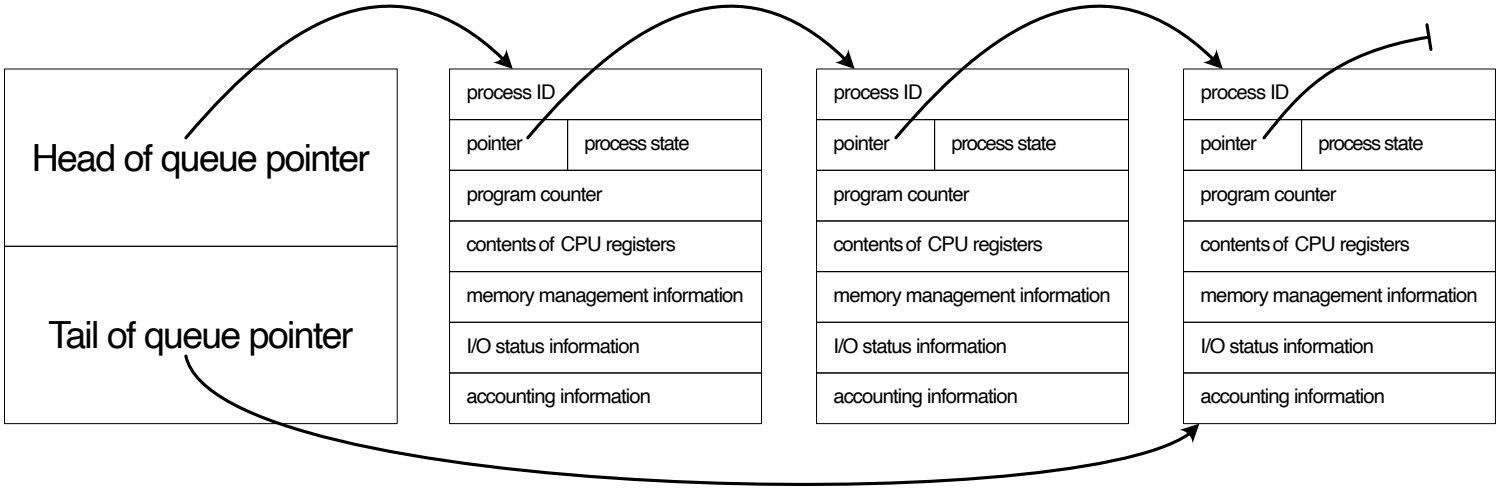
1. a process switches from running to waiting state;
2. a process switches from running to ready (due to an interrupt);
3. a process switches from waiting to ready state (completion of I/O);
4. a process terminates.

In a non-preemptive CPU scheduling system, scheduling takes place only within situations 1 and 4: the scheduler does not decide when to switch processes.

In a preemptive CPU scheduling system, the scheduler can intervene during situations 2 and 3 as well. A running process can be made to give way to other processes with a higher need of being scheduled.

1) First-come-first-served (FCFS) scheduling

The simplest method is to execute the processes in the ready queue on a first-come-first-served (FCFS) basis.



- When a process becomes ready, it is put at the tail of the queue.
- When the currently executing process terminates, or waits for I/O, the process at the front of the queue is selected next.

This algorithm is non-preemptive.

Gantt Charts for scheduling algorithm evaluation

The operation of a scheduling algorithm is commonly represented in a Gantt chart: time runs from left to right, and each process is represented by a block of time on the chart.

Consider the following initial CPU bursts for a set of processes:

Process	Arrival Time	Burst Time
P1	0	24
P2	1	3
P3	2	3

The Gantt chart for FCFS with the above data is:



Computing process waiting times from a Gantt chart

Process	Arrival Time	Burst Time
P1	0	24
P2	1	3
P3	2	3



Waiting times:

- P1? 0ms
- P2? $24 - 1 = 23$ms
- P3? $27 - 2 = 25$ms
- Average Waiting Time? $(0 + 23 + 25) \div 3 = 16$ms

Changing arrival order changes waiting times

Process	Arrival Time	Burst Time
P1	2	24
P2	0	3
P3	1	3



Waiting times:

- P1? $6 - 2 = 4$ms
- P2? 0ms
- P3? $3 - 1 = 2$ms
- Average Waiting Time? $(4 + 0 + 2) \div 3 = 2$ms.

FCFS: advantages and disadvantages

Advantages:

- Easy to implement.
- Easy to understand.

Disadvantages:

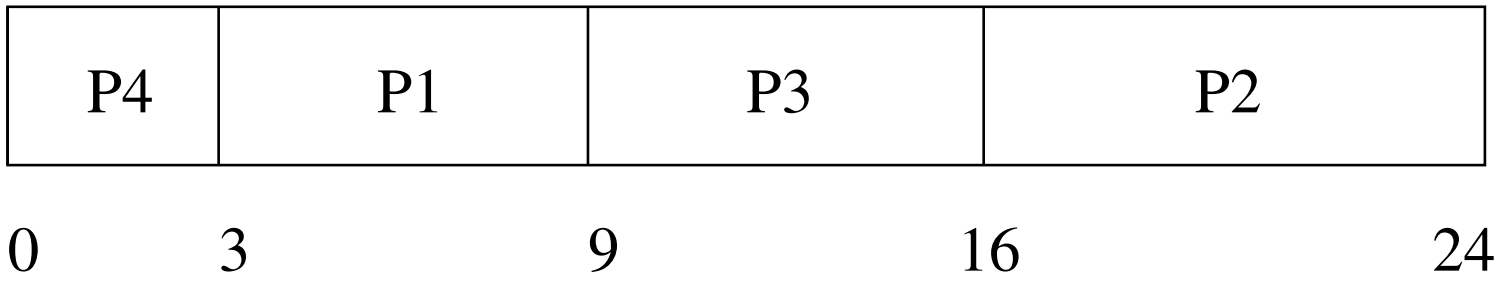
- Waiting time not likely to be minimal.
- Convoy effect: lots of small processes can get stuck behind one big one.
- Bad response time. (So, bad for time-sharing systems).

Q: Could throughput, i.e., the number of processes per time unit going through the system, be improved?

2) Shortest-Job-First (SJF) scheduling

If we knew in advance which process on the list had the shortest burst time, we could choose to execute that process next. This is called shortest-job-first (SJF) scheduling. For example:

Process	Burst Time
P1	6
P2	8
P3	7
P4	3



N.B. processes with equal burst times are executed in FCFS order.

SJF: advantages and disadvantages

Advantages:

- Provably optimal average waiting time.

Disadvantages:

- Length of the next CPU burst typically not known in advance.
- There is the possibility of long processes never getting executed.

Predicting the length of the next CPU burst

It is possible to approximate the length of the next CPU burst:

- it is likely to be similar in length to the *previous* CPU bursts.

A commonly-used formula: the exponential average CPU burst length.

$$\tau_{n+1} = \alpha\ t_n + (1 - \alpha)\tau_n$$

where:

- τ_n is the predicted length of CPU burst n.
- t_n is the actual length of CPU burst n.
- α is a value between 0 and 1 to adjust the weight given to the most recent CPU burst time.

Preemption and SJF scheduling

Scenario:

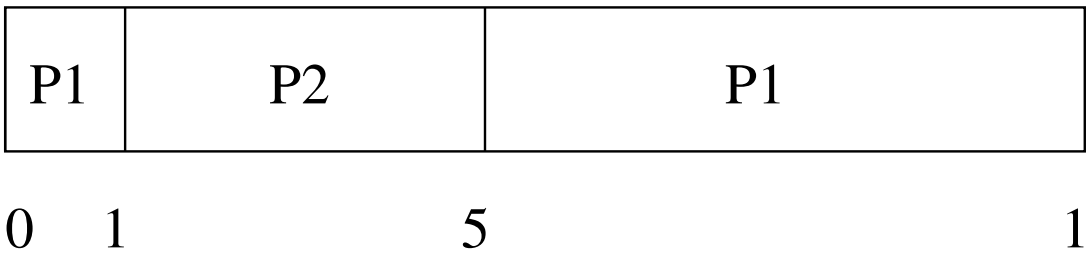
- A process P1 is currently executing.
- A new process P2 arrives before P1 is finished.
- P2's burst time is shorter than the *remaining* burst time of P1.

Contrast the alternatives:

- Non-preemptive SJF: P1 keeps the CPU until it completes.
- Preemptive SJF: P2 is given the CPU.

Tiny example:

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4



3) Priority Scheduling

In priority scheduling,

- each process is allocated a priority when it arrives;
- the CPU is allocated to the process with highest priority.

Priorities are represented by numbers: low number $\rightarrow$ high priority.

Priority scheduling can be preemptive.

We have actually already seen an example of priority scheduling: SJF.

- The priority of a process in SJF is set to be the estimated next CPU burst.

We can solve ‘starvation’ using ‘aging’

Starvation occurs when a process is having to wait persistently to be allocated to the CPU.

Priority scheduling algorithms are susceptible to starvation:

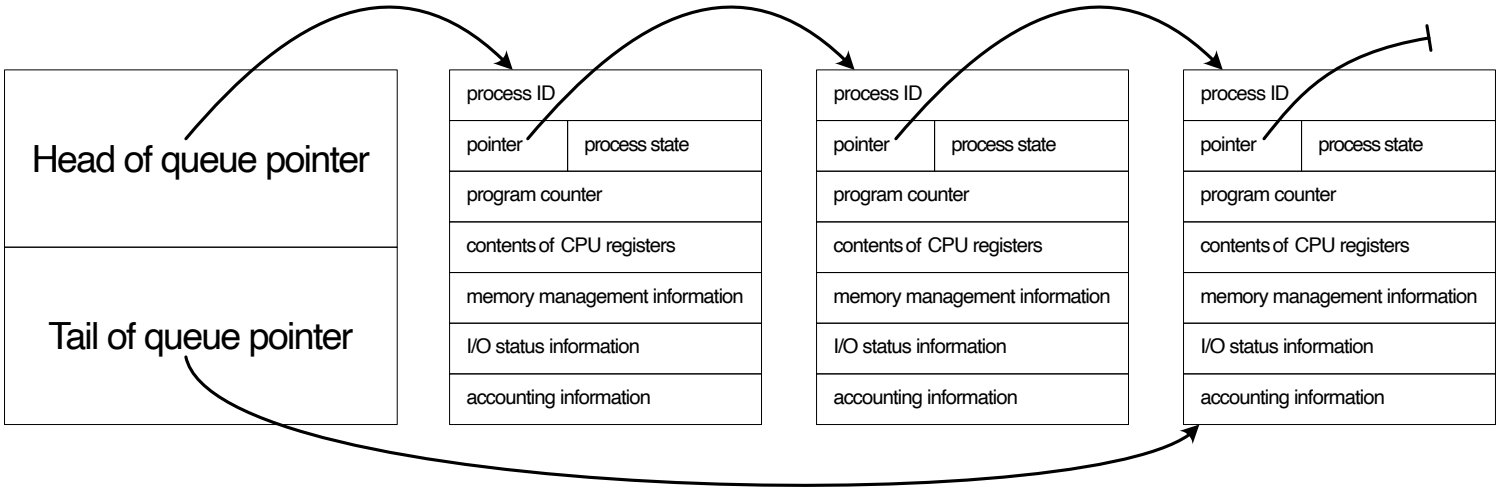
- Imagine a process P1 is waiting for the CPU, while a stream of higher-priority processes is arriving.
- If these processes arrive sufficiently quickly, P1 will never get a chance to execute.

A solution to scheduling starvation is to increase the priority of processes as a function of how long they have been waiting for the CPU—this is called aging.

4) Round-Robin Scheduling

Round-robin (RR) scheduling is designed for time-sharing systems.

- A small unit of time (time quantum) is defined.
- The ready queue is treated as a circular list.
- The CPU scheduler goes round the ready queue, allocating the CPU to each process for a time interval of *up to* 1 time quantum.

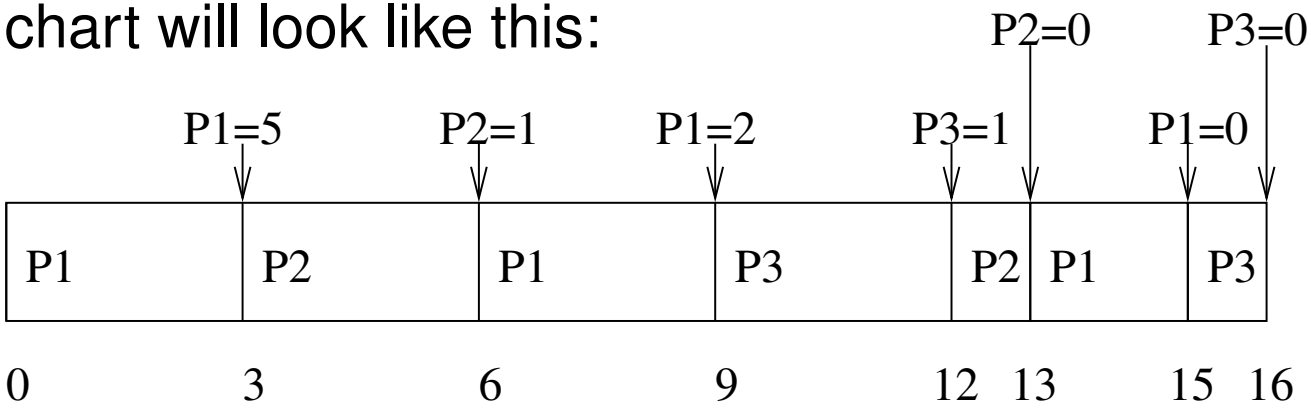


An example Gantt chart of Round-Robin scheduling

Say we have an RR algorithm with a quantum of 3, and the following process info:

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	4	4

The Gantt chart will look like this:



Note: in RR it is possible that a process *re-enters* the ready queue at the same time that another process arrives in the queue. FCFS is used as a tie-breaking mechanism.

Changing the time quantum in RR Scheduling

If the time quantum is set to be infinitely large:

- RR scheduling becomes FCFS scheduling.

If the time quantum is set to be very small:

- We get the effect of processor sharing.

However, there are drawbacks to making the time quantum very small:

- Consider setting the time quantum to be the same duration as the context switch time . . .
- . . . we will spend half our time on context switching!

RR Scheduling: advantages and disadvantages

Very much depends on the size of the time quantum.

- Large quantum: get advantages/disadvantages of FCFS.
- Very small quantum: get faster response time but slower throughput. (More time is spent context-switching.)
- Even ignoring context-switching time, turnaround time goes down if most processes complete within a single quantum.
  - e.g., say there are 3 processes with a next CPU burst of 10.
  - If the quantum size is 1, then average turnaround time is 29.
  - But if quantum size is 10, then each process finishes in the next CPU burst, and the average turnaround time is 20.
- Taking context-switching time into account, making the time quantum small has the effect of increasing turnaround time.

Readings

For this lecture, you should read Chapter 5 (Sections 1, 2, 3 and 7) of Silberschatz *et al*.